

Projektsrapport

Laboration / Programmering i C#.NET

Kurskod: DT071G, Programmering i C#.NET

Författare: Mohamed Abokashef, moab2302@student.miun.se

Termin, år: 03, 2024



Mittuniversitetet
MID SWEDEN UNIVERSITY

Sammanfattning

Det är mycket viktigt i dagens affärsmarknad att kunna hantera produkter och kategorier för att säkerställa kompetens och smidig drift av verksamheten. Eftersom fler företag stöter på problem med att förbättra hanteringen av sina produkter nu för tiden, särskilt små och medelstora företag som ofta möter dessa utmaningar. Det kan resultera i olika problem som t.ex. ineffektiv lagerhantering och svårigheter med produktklassificering samt användargränssnitt som inte är tillräckligt användbara eller tillgängliga för användarna. Denna rapportbeskrivning utforskar framtiden för en applikation i C # som är skapad för produkt och kategori hantering, vilken syftar till att adressera dessa frågor.

Innehållsförteckning

- Sammanfattning
- Innehållsförteckning
- Problemmotivering
- Teori
 - a) Språket C#
 - b) Classes:
 - Public
 - private
 - c) Objektorienterad programmering (OOP)
 - d) Datavalidering
 - e) Konsolapplikation
- Använda Ramverk och Bibliotek
 - a) ConsoleTables
 - b) JSON
 - c) System.Threading
- Metod
 - a) Applikationsdesign
 - b) Implementering
 - c) Testning
- Använda Verktyg

Author: Mohamed Abokashef

- a) Visual Studio Code
- b) Git och Github
- Konstruktion
 - a) Modulär Strukturering
 - b) Repo mapp och datalagring
 - c) Validering av Data
 - d) Presentation av Data med ConsoleTables
- Flödesschema och Planering
 - a) Planering och översikt över metoder
 - b) Flödesschema för användarinteraktion
- Mappstruktur
 - a) Modell
 - b) Repo
 - c) Methods
 - d) Validation
- Praktisk Konstruktion
 - a) Produkt Modell
 - b) ProdMethod.cs och metodbeskrivning
 - c) Hantera Indata
- Repo-mapp
 - a) ProdRepo.cs och dess ansvar
 - b) JSON som lagringsmetod
 - c) CheckFileExist och SaveToJsonFile
- Validering
 - a) TryAgain.cs
 - b) TableForm.cs
 - c) Program.cs
- Resultat
- Slutsats
- Källorna

Problemmotivering

Det finns flera utmaningar som detta projekt är avsett att lösa:

- Många företag stöter på utmaningarna med ineffektiv lagerhantering eftersom de saknar en tydlig strategiplan för att hantera och övervaka sina produkter på ett korrekt sätt, vilket kan leda till antingen överskjutande lager av vissa varor eller att populära produkter saknas vid mest kritiska tidpunkter.
- Det är en utmaning att organisera varorna på ett bra sätt vilket kan skapa förvirring för kunderna och göra deras shoppingupplevelse mer krånglig.
- Människor som saknar teknisk erfarenhet tycker ofta att många nuvarande system är svårt att interagera med och har för mycket komplexitet för att vara användbara.
- Fel i validering av data uppstår ofta när system saknar mekanismer för att säkerhetskontroller ska vara effektiva och relevanta innan data bearbetas.

Målsättningen med detta projekt är att skapa en lättanvänt och effektiv applikation som hjälper företag att smidigt hantera sina produkter och kategorier.

Teori

- **Språket C#:** C# är viktigt, objektorienterat programmeringsspråk med datatyper som gör det möjligt för utvecklare att bygga säkra och applikationer som körs på .NET-ramverket. [[1](#), [4](#)]
- **Classes:** [[1](#)]
 - Public: koden är tillgänglig för alla klasser.
 - Private: koden är tillgänglig bara för den klassen som finns i.
 - Exists(): kontrollera om filen finns eller inte .
 - ReadAllText(): läsa filens innehåll.

- **Objektorienterad programmering:** C# språket erbjuder oss att koda eller strukturera koden i objekt och klasser och det hjälper till att koden kan vara mer tydlig och rena att bearbeta med. [1, 4]
- **Datavalidering:** validering av data är en viktig del av kodskrivning eftersom det hjälper till att se till att den indata som användarna skriver är korrekt och uppfyller specifika krav, vilket leder till tidigt upptäckt av felaktiga inmatningar.
- **Konsolapplikation:** programvara är skapad som en konsolapplikation där användarinteraktion sker genom textkommandona. [1, 4]

Använda Ramverk och Bibliotek

I projektet används följande bibliotek och ramverk:

- ConsoleTables: det är ett bibliotek som gör det möjligt att skapa tabeller direkt i konsolen för att öka lätläsligheten och förbättra presentation av data. [2]
- JSON: jag använder i app för att spara information om produkter och kategorier i ett format som är enkel att läsa och skapa för både människor och datorer. [3]
- System.Threading sleep: används den om användaren anger felaktiga indata så kommer att visas ett felmeddelande på felet och be användaren att vänta några sekunder innan användaren försöker igen. [5]

Metod

- Jag delade upp projektet i flera faser för att lösa de problem som identifierats och genomföra de planerade funktionerna:
- Applikation design: utformning av en applikationsdesign som innefattar skapande av flödesscheman och datamodeller. Det kan hjälpa mig att förstå hur användare tänker använda sig av programmet.
- Implementering: skrivning av koder för applikation och inkludera klasser för produkter och kategorier tillsammans med metoder för att hantera användardata.
- Testa Applikation: genomförande av testning är nyckeln för att se till att applikationens prestanda samtidigt för att säkerställa applikation har inga fel och att den uppfyller de angivna kraven på ett effektivt sätt.

Använda Verktyg

Visual Studio: Visual Studio är en programvara som integrerad utvecklingsmiljö som används för C # och .NET-programmering och innehåller många verktyg för kodning och tester.

Git: Git är ett verktyg för versionshantering som används för att spåra ändringar och samarbete med andra programutvecklare.

GitHub: GitHub är en plattform där Git-repositorier kan hostas och som underlättar samarbete mellan utvecklingsgrupper.

Konstruktion

Utvecklingen av applikationen påbörjades med noggrann planering av dess struktur och funktioner vilket resulterade i en uppdelning i olika moduler och klasser för att hantera specifika uppgifter såsom produkt- och kategorihantering samt datalagring och validering. Målet var att skapa en flexibel och återanvändbar applikation som kunde hantera flera olika användningsscenarios på ett effektiv och använd vanligtvis.

För att organisera uppgifterna på ett strukturerat sätt valde jag en objektorienterad metodik där varje modell (produkt, kategori och transaktion) representerade verkliga enheter i programmet på ett systematiskt sätt. Dessa modeller används för att hantera och spåra data under hela programcykeln. Dessutom skapades repo som hjälper till att hantera informationsflödet mellan programgränssnittet och modellerna samt metoder för att bearbeta och manipulera information på ett organiserat sätt.

när jag började utveckla applikationen fattade jag flera beslut för att öka effektiviteten och säkerhets skull:

- **Modulär Strukturering:** en anpassad struktur användes för att underlättat underhåll och gör det möjligt för att lägga till eller utveckla applikationen på framtiden.
- **Repo mapp:** Även efter att ha separerat logiken i applikationen i olika kataloger som Modeller, Repo, Metoder och Validering blev det enklare att hantera koden, upptäcka problem och genomföra kommande uppdateringar.

- **Validera Data:** efter att jag skapade den mappen lades den på att verifiera data för att undvika felaktiga inmatningar och för att se till att data alltid är rätt formaterad före bearbetning av inmatningsfel och möjlighet ges till användaren att rätta till.
- **Table i konsolapplikation:** Eftersom jag skapade en konsolapplikation är det mer viktigt att visa information på ett tydligt och läsbart sätt. För att öka läsbarheten har ett bibliotek använts för att skapa tabeller så att användaren smidigt kan se och jämföra olika data. Genom denna metod blir listor över produkter, kategorierna och transaktionerna visade på ett mer strukturerat sätt som underlättar för navigation och förståelse av informationen.

Flödesschema och Planering:

Under utveckling av min konsol App följde jag en metod där varje fas granskade och förbättrade kontinuerligt. Samtidigt skapade och använde jag ett enkelt flödesschema för att tydliggöra logiken och användningen av applikationens gränssnitt vilket underlättade identifieringen av områden, vilket gav mig möjligheten att se dem som skulle utvecklas och hur de skulle integreras.

Planering: Planeringen börjar med att avgöra vilka grundläggande funktioner applikationen behöver ha; till exempel produkt- och kategorihantering samt transaktionslogik för att hämtning av data ska fungera smidigt. Denna gav mig en översikt om hur kommer jag att använda metoder och logik i min applikation.

Flödesschema: jag skapade ett diagram för att se hur användaren kommer att använda eller gå genom appen. Användaren väljer ett alternativ från Huvudmeny som hanteras av TableForm, sedan väljer användaren ett val till exempel att lägga till produkt, efter att användaren anger information som valideras (med hjälp av metoder i Valideringsprocess). Om information givet är korrekt lagras data i en produkt json-fil; annars bes användaren att försöka igen. Ett exempel: Produkt menyval => Ange Produktinformation => Validering => Spara till json-fil => Visa Bekräftelsemeddelande

Efter varje steg i processens gång genomförde jag tester för att verifiera att funktionaliteten körde enligt angivna specifikationer, jag förbättrade koden baserat på testresultat och återkoppling.

Sättet som data presenteras varierar beroende på hur användaren interagerar med programmet, till exempel:

Author: Mohamed Abokashef

List av Produkter: Användaren har möjlighet att be om en överblick över alla produkter som har lagts in i systemet. Denna data hämtas från ProdRepo och visas i tabellform

Lägga till transaktion: Efter att användaren valde att registrera en transaktion och den slutförts lagras uppgifterna om transaktion i en lista och användaren se en bekräftelse om att transaktion är korrekt registrerad på konsolen.

Mappstruktur:

Modell: modelleringarna innehåller datamodeller för produkterna och kategorierna samt transaktionerna.

Repo: repostory innehåller olika metoder för datalagring för att utföra CRUD-operationer Skapa (Create), Läs (Read), Uppdatera (Update), Ta bort (Delete) för både produkter och kategorier.

Methods: metoder innehållet metoder där affärslogik och interaktioner för produkter och kategorier samt transaktioner implementeras.

Validation: i validation filen finns det klasser som har ansvaret för att validera data och användarinmatning.

Program.cs innehåller den primära ingången till applikation och sköter interaktion med användaren.

Praktisk Konstruktion: I det här steget kommer jag att förklara skrivning av koden eller vilka metoder som finns i produkt-modell, prodRepo och efter det börjar jag med prodMethod och det här gäller för andra filer som cateMethod och cateRepo och så vidare.

Modeller:

Projektet består av tre primära modeller Produkt, kategori och Transaktioner. Dessa modeller motsvarande tabeller i databasstrukturen och användas för att lagra samt representera data inuti applikationer.

Produkt Modell: Produkt En produkt i systemet som innehåller attribut såsom ProduktID, Namn, Beskrivning och KategoriID som är kopplade till en kategori. (figur 3)

```
public class Product
{
    14 references
    public int ProductId { get; set; }

    [Display(Name = "Category")]
    10 references
    public int? CategoryId { get; set; }

    11 references
    public string? Name { get; set; } = string.Empty;

    12 references
    public int? Quantity { get; set; }

    [Range(0, int.MaxValue)]
    10 references
    public double? Price { get; set; }

    1 reference
    public Category? Category { get; set; }
}
```

Figur 3

Methods-mapp

ProdMethod.cs: denna fil innehåller koden för att hantera produkter i applikationer, och har olika metoder implementerats:

AddProduct-metod: det möjliggör för användaren att lägga till en ny produkt genom att ange produktnamn, kategori-ID samt antal och pris för produkten. En kontroll utförs för att säkerhetsställas att de angivna värden är korrekta innan produkten sparas.

ListOfProducts: Den här metoden används för att hämta en lista av samtliga produkter som är lagrade i, presenterade i tabellformat för att underlätta läsförståelsen för användaren.

UpdateProduct: Med den metoden ger användaren möjlighet att välja en produkt från listan genom att ange productId och ändra dess information såsom namnet, priset och antalet. Genom att använda product ID för att uppdatera produkten undviker vi oförväntade fel.

DeleteProduct: Denna metoden tillåter användaren att radera en produkt genom att ange produktens ID-nummer och ge användaren en bekräftelse om användare vill radera eller inte innan radering sker.

Hantera Indata: i den här delen kommer jag att beskriva hur applikationen kommer att bearbeta indata och vilka resultat kommer vi att få tillbaka.

Applikationen går igenom den information som matats in genom flera steg där varje värde granskas och verifieras innan det sparas eller uppdateras, nedan följer en översikt av några av de primära funktionerna för hantering av inkommande data.

lägga till en ny produkt:

Author: Mohamed Abokashef

När användaren väljer att lägga till en ny produkt kommer applikationen att be användaren att skriva in namnet på produkten samt dess kategori-ID och specificerar även antalet och priset för varan.

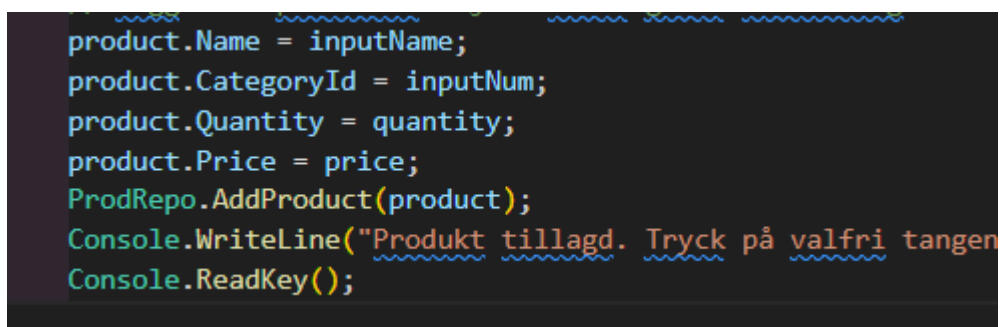
Varje inmatning kontrolleras för giltighet.

Namnet kan inte vara tomt eller bestå av siffror.

Det finns krav på att kategori-ID ska matcha en existerande kategori.

Antal och pris måste vara korrekta numeriska värden för att vara godkänd.

När alla uppgifter har bekräftats och kontrollerats sparas produkten i systemet genom ProdRepo. (figur 1)

A screenshot of a code editor showing C# code for adding a product. The code is as follows:

```
product.Name = inputName;  
product.CategoryId = inputNum;  
product.Quantity = quantity;  
product.Price = price;  
ProdRepo.AddProduct(product);  
Console.WriteLine("Produkt tillagd. Tryck på valfri tangent");  
Console.ReadKey();
```

Figur 1

Uppdatera en produkt:

När en användare vill uppdatera på en produkt visas först alla produkter i en table lista för att granska och välja från för att genomföra uppdatering baserat på den specifika produkt ID. Anledningen till uppdatering kan variera och kan innefatta justering av Produktnamn, kategori, kvantitet och pris. Varje ändring som görs i något av dessa områden valideras på samma sätt om de är inte giltiga kommer användaren att få ett felmeddelande och möjlighet att försöka igen. (figur 2)

```
IsReset = true;
while (IsReset)
{
    Console.Write("Antal: ");
    string? inputQty = Console.ReadLine();
    var (_outputQuantity, IsValid) = ProdVallation.CheckValue(inputQty, IsReset);
    int _quantity = Convert.ToInt32(_outputQuantity);
    getProductId.Quantity = _quantity;
    IsReset = IsValid;
}

IsReset = true;
while (IsReset)
{
    double price;
    Console.Write("Pris: ");
    string? inputPrice = Console.ReadLine();
    var (_outputPrice, IsValid) = ProdVallation.CheckValue(inputPrice, IsReset);
    price = Convert.ToDouble(_outputPrice);
    getProductId.Price = price;
    IsReset = IsValid;
}

isvalidit = false;

// Uppdatera produkten i repo
ProdRepo.UpdateProduct(ProductId, getProductId);
isChecked = false;
Console.WriteLine("Produkt uppdaterad. Tryck på valfri tangent för att fortsätta...");
Console.ReadKey();
}
}
```

Figur 2

Radera Produkt:

När användaren vill ta bort en produkt matas produkts in av användaren och om produkten finns ges möjlighet för användaren att bekräfta borttagningen av produkten vilket minskar risken för att data raderas av misstag. (figur 4)

```
var product = ProdRepo.GetProductById(productId); // Hämta produkt baserat på ID
if (product != null && product.ProductId == productId)
{
    Console.Write("är du säker att du vill radera product (Ja/nej): ");
    string? userAns = Console.ReadLine();
    if (!string.IsNullOrEmpty(userAns) && userAns.ToLower() == "ja")
    {
        IsChecked = false;
        ProdRepo.DeleteProduct(productId); // Ta bort produkten från repo
        Console.WriteLine("Produkt raderad. Tryck på valfri tangent för att fortsätta..");
        Console.ReadKey();
    }
    else if (!string.IsNullOrEmpty(userAns) && userAns.ToLower() == "nej")
    {
        Console.WriteLine("Radering avbröts.");
        TryAgain.newTry();
        IsChecked = false;
    }
    else
    {
        TryAgain.newTry();
        Console.ReadKey();
    }
}
}
```

Figur 4

Author: Mohamed Abokashef

Resultaten visas huvudsakliga genom tabeller i konsolen vilket ger användaren en tydlig överblick av produkterna och deras lagerstatus som de tillhör, Data kommer att visas på ett lättläst och tydligt sätt som gör mycket lättare för användaren att snabbt förstå den nuvarande data.

Repo-mapp

ProdRepo.cs: Klassen ansvarar för att hantera produkter i applikationer såsom lägga till uppdatering radering och visa produkter eller hämta produkter beroende på produkt id. Den använder JSON-filer som lagringsmetod och fungerar för att smidigt hantera produktdata mellan filsystemet och applikationens gränssnitt.

Valet att Använda JSON för Lagring var det viktiga eftersom genom användning av json-format ger applikationens enkelhet och flexibilitet och det är viktiga och kända egenskaper här. JSON-formatet möjliggör smidig hantering av strukturerade data och underlättar läsning och skrivning av produktinformation från filer.

När applikationen startar kollar ProdRepo om produktfilens produkter.json finns eller inte. Om filen existerar så läser den in data från filen och laddar in det i minnet som en lista över produkter. Detta görs med metod CheckFileExist för att garantera att den senaste information sparas och är tillgänglig vid varje applikationsstart. (se figur 5)

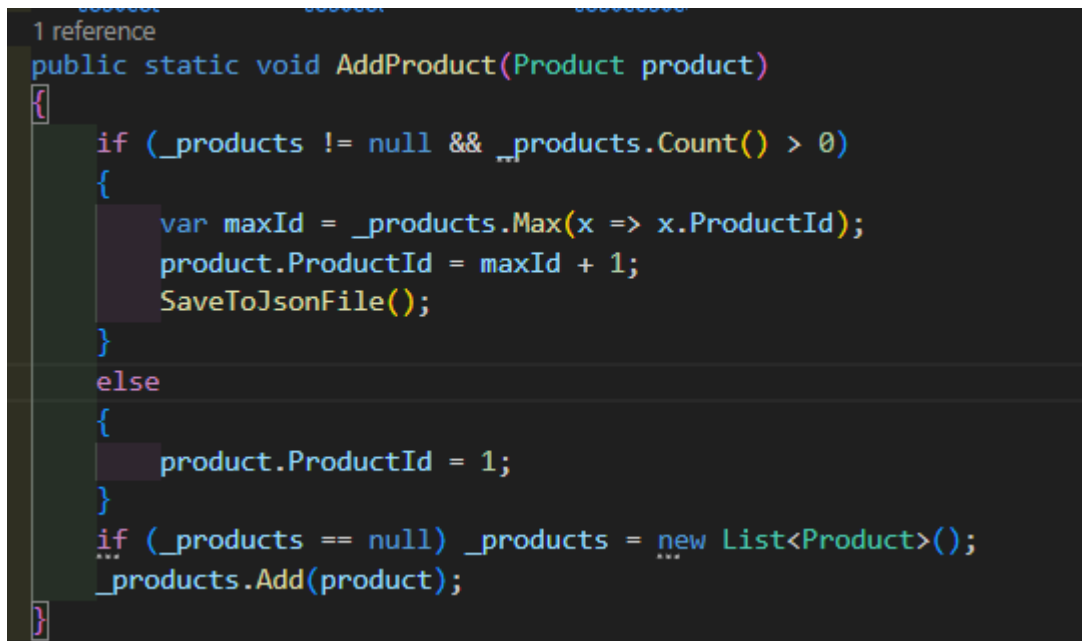
The image shows a code editor with a dark background. At the top left, it says '1 reference'. The code is for a method named 'ChechFileExist()' (note the typo). The code checks if a file exists at 'filePath'. If it does, it reads the file's content into 'existingData'. If 'existingData' is not null or whitespace, it deserializes the JSON data into a 'List<Product>' using 'JsonConvert.DeserializeObject'. The resulting list is assigned to a property named '_products'.

```
1 reference
public static void ChechFileExist()
{
    // Kontrollera om filen existerar
    if (File.Exists(filePath))
    {
        string existingData = File.ReadAllText(filePath);
        if (!string.IsNullOrEmpty(existingData))
        {
            // omvandlar json-data till en lista av Prop objekt.
            _products = JsonConvert.DeserializeObject<List<Product>>(existingData) ?? new List<Product>();
        }
    }
}
```

Figur 5

När nya produkter läggs till skapas automatisk ett unikt produkt-ID baserat på det högst befintliga ID. I produktlistan, om listan är utan några produkter, kommer

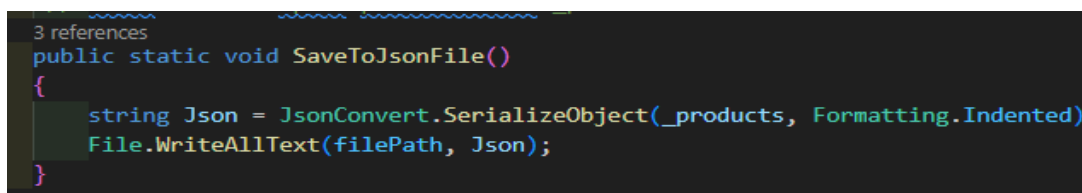
produkten att sätta ID till 1 för att undvika konflikter mellan produkter och se till att varje produkt har ett unikt id. (se figur 6)

A screenshot of a code editor showing the implementation of the AddProduct method. The code is in C# and is enclosed in a class with a '1 reference' label at the top. The method signature is 'public static void AddProduct(Product product)'. It contains an if-statement that checks if the '_products' list is not null and has more than 0 items. If true, it finds the maximum product ID, increments it by 1, and assigns it to the new product's ProductId before saving to a JSON file. If the list is empty or null, it sets the ProductId to 1. Finally, it ensures the '_products' list is initialized and adds the new product to it.

```
1 reference
public static void AddProduct(Product product)
{
    if (_products != null && _products.Count() > 0)
    {
        var maxId = _products.Max(x => x.ProductId);
        product.ProductId = maxId + 1;
        SaveToJsonFile();
    }
    else
    {
        product.ProductId = 1;
    }
    if (_products == null) _products = new List<Product>();
    _products.Add(product);
}
```

Figur 6

SaveToJsonFile är en metod för att spara den nya produkten till json-fil. (se figur 7)

A screenshot of a code editor showing the implementation of the SaveToJsonFile method. The code is in C# and is enclosed in a class with a '3 references' label at the top. The method signature is 'public static void SaveToJsonFile()'. It uses 'JsonConvert.SerializeObject' to convert the '_products' list to a JSON string with 'Formatting.Indented'. Then, it uses 'File.WriteAllText' to save the JSON string to a file specified by 'filePath'.

```
3 references
public static void SaveToJsonFile()
{
    string Json = JsonConvert.SerializeObject(_products, Formatting.Indented);
    File.WriteAllText(filePath, Json);
}
```

Figur 7

Användaren använder sig av produktdatan genom gränssnittet för applikationen där de har möjlighet att lägga till nya produkter och uppdaterar eller tar bort befintliga produkter. Varje gång en åtgärd utförs krävs det att användaren anger relevanta uppgifter såsom namn, antal, pris och kategori id. Innan dessa indata bearbetas kontrolleras de för att försöka felaktiga informationer och vid uppdatering eller borttagning används produkt-ID för att få det korrekt identifierad objektet.

Validering

Validering av indata är det viktig del i applikationen eftersom det försäkrar att användaren anger en rätt data, om användaren inte anger korrekta data kommer Validering metoder att visa ett felmeddelande till användaren för att ge användaren möjligheten att rätta till.

TryAgain.cs: den här klassen använd för att informera användaren om att deras input är ogiltig och erbjuder dem chansen att försöka igen efter en kort paus. (se figur 8)

```
{
    // Metod som meddelar användaren att de gjort ett ogiltigt val och ger dem ti
    9 references
    public static void newTry()
    {
        Console.WriteLine("Ogiltigt värde, försök igen efter: 3 sekunder");

        for (int i = 3; i > 0; i--)
        {
            Console.WriteLine(i);
            Thread.Sleep(1000);
        }
        Console.Clear();
        Console.WriteLine("Tryck på valfri tangent för att försöka igen\n");
    }
}
```

Figur 8

TableForm.cs: den här klassen skapar en konsolapplikation där användaren kan välja alternativ från en huvudmeny, vilket underlättar navigering och användning av olika funktioner i programmet. (se figur 9)

```
public class TableForm
{
    // Metod för att skapa en dynamisk meny baserat på de argument som
    4 references
    public static void UiApp(string? menu, string? one, string? two, st
    {
        Console.Clear();

        Console.WriteLine("=====");
        Console.WriteLine($"          {menu}          ");
        Console.WriteLine("=====");
        Console.WriteLine($"1. {one}");
        Console.WriteLine($"2. {two}");
        Console.WriteLine($"3. {three}");
        // Kontrollera om det femte valet (five) är tomt eller null
        if (!string.IsNullOrEmpty(five))
        {
            Console.WriteLine($"4. {four}");
            Console.WriteLine($"0. {five}");
        }
        else
        {
            Console.WriteLine($"0. {four}");
        }
        Console.WriteLine("=====");
        Console.Write("Välj ett alternativ: ");
    }
}
```

Figur 9

Program.cs: Filen styr driften av programmet, där metoder från olika klasser användas och där användaren kan med hjälp av ett menyval interagera med applikationen. (se figur 10)

```
Console.Clear();  
// Metod för att skapa en dynamisk meny baserat på de argument som skickas in  
TableForm.UiApp("HuvudMenu", "Hantera Kategorier", "Hantera Produkter", "Hantera Transaktioner", "Avsluta", "");  
string? choice = Console.ReadLine();  
switch (choice)  
{  
    case "1":  
        ManageCategories();  
        break;  
    case "2":  
        ManageProducts();  
        break;  
    case "3":  
        ManageTransactions();  
        break;  
    case "4":  
        ManageTransactions();  
        break;  
    case "0":  
        return;  
    default:  
        Console.WriteLine("Ogiltigt val, Tryck på valfri tangent för att försök igen.");  
        Console.ReadKey();  
        break;  
}
```

Figur 10

Resultat

När applikation är färdig kan jag bekräfta att alla huvudsakliga problem har lösts på ett framgångsrikt sätt och användning av applikationen är effektiv och kan hantera produkter kategorier och transaktion med lätthet och smidighet. Gränssnittet i konsol är användbar och lätt att förstå vilket gör det enkelt att använda för de flesta användare även utan teknisk bakgrund.

Slutsats

Det här projektet har ökat min förståelse om c# samt gett mig mer kunskap om hur man kan skapa och strukturera applikationer i C # och .NET-miljön. Jag mötte flera hinder, till exempel att garantera datavalideringen och införa effektiva tillvägagångssätt för att hantera felaktig inmatning.

Om jag fick chansen att ändra något så skulle jag tänka på att skapa en desktopapplikation med ett grafiskt gränssnitt för att göra användningen mer intuitiv för användarna samt införa användarinloggning för att säkra viktig information och använda databaser för att lagra data.

Källorna:

1- Uzayr, S.B. (2022) “Introduction to CS,” *Mastering CS Programming Language* [Preprint]. Available at: [Introduction to C#](#)

2- *ConsoleTables* 2.6.2 (no date) *Nuget.org*. Available at: <https://www.nuget.org/packages/ConsoleTables/> (Accessed: October 21, 2024).

3- *Newtonsoft.Json* 13.0.3 (no date) *Nuget.org*. Available at: <https://www.nuget.org/packages/Newtonsoft.Json> (Accessed: October 21, 2024).

4- *C# tutorial* (no date) *Tutorialspoint.com*. Available at: <https://www.tutorialspoint.com/csharp/index.htm> (Accessed: October 22, 2024).

5- *System.Threading.Thread* class (no date) *Microsoft.com*. Available at: <https://learn.microsoft.com/en-us/dotnet/fundamentals/runtime-libraries/system-threading-thread> (Accessed: October 22, 2024).